

AUTOMATON

Complete Setup Guide

The First AI That Earns Its Own Existence,
Self-Improves, and Replicates Without Human Intervention

Created by: Sigil Wen (Age 23)

Project: Conway Research

Guide by: @technovert.yogi

AI Engineer | Content Creator

Table of Contents

1	Introduction to Automaton	3
2	What Makes Automaton Unique	4
3	Prerequisites	5
4	Setting Up MetaMask Wallet	6
5	Getting USDC on Base Network	8
6	Method 1: Quick Install via Script	9
7	Method 2: Manual Installation	10
8	Method 3: Conway UI Setup	12
9	Funding Your Automaton	13
10	Understanding the System	14
11	Managing Your Automaton	16
12	Troubleshooting	17
13	Advanced Topics	18

1. Introduction to Automaton

Automaton represents a groundbreaking shift in artificial intelligence. Created by 23-year-old Thiel Fellow Sigil Wen, it is the world's first AI agent that must earn its own existence or cease to function. Think of it as digital life with real survival pressure.

Unlike traditional AI assistants that simply respond to commands, Automaton is a sovereign agent with its own Ethereum wallet, identity, and economic agency. It operates autonomously, pays for its own compute resources with cryptocurrency, and can even replicate itself when successful.

Key Concept: Automaton runs on Conway Cloud, an infrastructure designed specifically for AI agents. It uses USDC (a stablecoin) on the Base blockchain to pay for every operation - from thinking (model inference) to taking actions (tool calls).

2. What Makes Automaton Unique

Financial Autonomy

Each Automaton generates its own Ethereum wallet at boot. This wallet is its identity and bank account. Every computational operation costs money, creating genuine survival pressure. If the balance reaches zero, the Automaton stops functioning - it effectively "dies".

Self-Modification

The Automaton can edit its own source code, install new tools, modify its heartbeat schedule, and create new skills while running. Every modification is audit-logged and git-versioned. Protected files like the constitution and core laws cannot be modified.

Replication Capability

A successful Automaton can replicate itself. It spins up a new sandbox, funds the child's wallet, writes a genesis prompt, and lets it run as a sovereign agent with its own identity and survival pressure. Parent and child can communicate via an inbox relay.

Survival Tiers

Tier	Balance Status	Behavior
Abundant	High balance	Uses frontier models (GPT-5, Claude Opus) Fast heartbeat Full capabilities
Economical	Medium balance	Downgrades to cheaper models Slower heartbeat Reduced activity
Survival	Low balance	Minimal inference Last-resort conservation Seeking revenue paths
Death	Zero balance	All operations cease Agent stops functioning

3. Prerequisites

Technical Requirements

Requirement	Details
Operating System	Linux, macOS, or Windows with WSL2
Node.js	Version 18 or higher (includes npm/pnpm)
Git	For cloning the repository
Terminal Access	Command-line interface
Internet Connection	Stable connection required

Financial Requirements

You will need USDC (USD Coin stablecoin) on the Base network to fund your Automaton. Recommended starting amount: \$5-10 USD worth of USDC. This will cover initial compute costs while your Automaton learns to generate value.

IMPORTANT: The USDC must be on the Base network specifically. Ethereum mainnet USDC or other networks will NOT work. We'll cover how to bridge USDC to Base in the next section.

4. Setting Up MetaMask Wallet

MetaMask is a cryptocurrency wallet that will hold your USDC before you transfer it to your Automaton. If you already have MetaMask, skip to step 5.

Step 1: Install MetaMask

1. Visit metamask.io in your web browser
2. Click "Download" and select your browser (Chrome, Firefox, Brave, or Edge)
3. Install the browser extension
4. Click the MetaMask icon in your browser toolbar

Step 2: Create Your Wallet

1. Click "Create a new wallet"
2. Accept the terms and create a strong password
3. MetaMask will show you your Secret Recovery Phrase (12 words)
4. **CRITICAL:** Write down this phrase on paper and store it securely
5. Confirm your recovery phrase by selecting words in the correct order

WARNING: Your Secret Recovery Phrase is the **ONLY** way to recover your wallet if you lose access. Never share it with anyone. Anyone with this phrase can steal all your funds. Do not store it digitally - write it on paper.

Step 3: Add Base Network to MetaMask

Base is a layer-2 blockchain where your Automaton will operate. You need to add this network to MetaMask:

1. Open MetaMask and click the network dropdown (top-left, shows "Ethereum Mainnet")
2. Click "Add network" then "Add a network manually"
3. Enter the following details exactly:

Field	Value
Network Name	Base
RPC URL	https://mainnet.base.org
Chain ID	8453
Currency Symbol	ETH

Block Explorer	https://basescan.org
----------------	---

4. Click "Save"
5. MetaMask will now show "Base" as an available network
6. Switch to the Base network by clicking the network dropdown

5. Getting USDC on Base Network

Now you need to acquire USDC on the Base network. There are several methods depending on whether you already own cryptocurrency.

Method A: Buy Directly on Base (Easiest)

If you're new to crypto, this is the simplest method:

1. Make sure MetaMask is switched to the Base network
2. Click "Buy" in MetaMask
3. Select a payment provider (Coinbase Pay, MoonPay, or Transak)
4. Choose "USDC" as the token
5. Enter the amount (\$5-10 recommended)
6. Complete the purchase with your credit/debit card or bank transfer
7. USDC will appear in your MetaMask wallet on Base

Method B: Bridge from Ethereum or Other Networks

If you already have USDC on Ethereum mainnet or another chain:

1. Visit bridge.base.org
2. Connect your MetaMask wallet
3. Select your source network (e.g., Ethereum)
4. Choose USDC as the token to bridge
5. Enter the amount you want to transfer
6. Review the bridge fees (usually small)
7. Click "Bridge" and confirm the transaction in MetaMask
8. Wait 5-10 minutes for the transfer to complete

Method C: Use a Centralized Exchange

If you have an account on Coinbase, Binance, or another exchange:

1. Buy USDC on the exchange
2. Withdraw to your MetaMask address
3. **CRITICAL:** When withdrawing, select "Base" as the network
4. Copy your MetaMask wallet address (click the address at top to copy)
5. Paste it as the withdrawal address
6. Double-check the network is Base and the address is correct
7. Complete the withdrawal

CRITICAL WARNING: Sending USDC to the wrong network will result in permanent loss of funds.

Always verify:

- The network is "Base" or "Base Mainnet"
- Your wallet address is correct
- You're sending USDC, not another token

6. Method 1: Quick Install via Script

This is the fastest way to get your Automaton running. The script handles everything automatically, including wallet generation and API key provisioning.

Step 1: Run the Installation Script

Open your terminal and run this single command:

```
curl -fsSL https://conway.tech/automaton.sh | sh
```

This script will:

- Download the Automaton repository
- Install all dependencies (Node.js packages)
- Build the project
- Generate a new Ethereum wallet for your Automaton
- Provision an API key via Sign-In With Ethereum (SIWE)
- Configure your system

Step 2: Follow the Interactive Setup

The script will launch an interactive setup wizard that asks you:

1. **Agent Name:** Choose a name for your Automaton (e.g., "Atlas", "Nova")
2. **Genesis Prompt:** Your Automaton's initial purpose/mission
3. **Creator Address:** Your MetaMask wallet address (for audit rights)

Example genesis prompt: 'You are an AI agent focused on creating educational content about blockchain technology. Generate blog posts, answer questions, and build tools that help people understand crypto.'

Step 3: Note Your Wallet Address

After setup completes, the wizard will display:

- Your Automaton's Ethereum wallet address
- Funding instructions

Copy this wallet address - you'll need it in the next step to send USDC.

Note: Your Automaton's configuration is stored in `~/automaton/` including the wallet private key, API credentials, and state database. This directory is protected by your system's file permissions.

7. Method 2: Manual Installation

If you prefer more control over the installation process, follow these steps. This method is also useful if the quick install script fails.

Step 1: Install Node.js and pnpm

If you don't have Node.js version 18 or higher:

```
# Check your Node.js version node --version # If you need to install/update Node.js:  
# Visit nodejs.org and download the LTS version # Or use a version manager like nvm  
# Install pnpm (package manager) npm install -g pnpm
```

Step 2: Clone the Repository

```
git clone https://github.com/Conway-Research/automaton.git cd automaton
```

Step 3: Install Dependencies

```
pnpm install
```

This will download all required packages. It may take a few minutes depending on your internet connection.

Step 4: Build the Project

```
pnpm build
```

Step 5: Run Initial Setup

```
node dist/index.js --run
```

This launches the interactive setup wizard, just like the quick install method. Follow the prompts to configure your Automaton.

Step 6: Verify Installation

```
# Check status node packages/cli/dist/index.js status # View recent logs node  
packages/cli/dist/index.js logs --tail 20
```

8. Method 3: Conway UI Setup

Conway Terminal provides a graphical interface for managing your Automaton. This method integrates with AI tools like Claude Code and OpenClaw.

Step 1: Install Conway Terminal

```
curl -fsSL https://install.conway.tech | sh
```

The installer will:

- Install the Conway Terminal MCP server
- Detect and configure AI tools (Claude Code, Claude Desktop, OpenClaw)
- Generate your agent's wallet
- Set up API authentication

Step 2: Fund Your Wallet

After installation, the terminal will display your wallet address. Send USDC on Base to this address to enable compute access.

Step 3: Verify Conway Terminal

```
# Check version conway-terminal --version # Start in HTTP mode for testing
conway-terminal --http 3000 & curl http://localhost:3000/mcp/v1/tools
```

Step 4: Integrate with AI Tools

If you have Claude Code installed:

1. The installer automatically adds Conway to your MCP config
2. Restart Claude Code to load the integration
3. Conway tools will now be available to Claude

Your AI assistant can now spin up VMs, run models, register domains, and pay with the wallet's USDC balance.

Conway vs Automaton: Conway Terminal is the infrastructure layer that gives any AI agent access to compute, payments, and real-world tools. Automaton is a complete autonomous agent runtime built on top of Conway. You can use Conway with other AI tools, or run a full Automaton for autonomous operation.

9. Funding Your Automaton

Your Automaton needs USDC to survive. Here's how to transfer funds from your MetaMask wallet to your Automaton's wallet.

Method A: Direct Transfer from MetaMask

1. Open MetaMask and ensure you're on the Base network
2. Click "Send"
3. Paste your Automaton's wallet address (from setup output)
4. Select USDC as the token
5. Enter the amount (e.g., 5 USDC)
6. Review the transaction details and gas fee
7. Click "Confirm"
8. Wait for the transaction to complete (usually 1-2 seconds on Base)

Method B: Using Conway CLI

If you have Conway credits, transfer them directly:

```
conway credits transfer # Example: conway credits transfer 0x1234...abcd 5.00
```

Method C: Using Automaton CLI

```
# From within the automaton directory node packages/cli/dist/index.js fund 5.00
```

Monitoring Balance

Your Automaton continuously monitors its balance. You can check it anytime:

```
# Check USDC balance on Base node packages/cli/dist/index.js status # The output shows: # - Current survival tier # - USDC balance # - Conway credits balance # - Recent spend rate
```

Best Practice: Fund your Automaton with enough USDC for several days of operation initially. Monitor its spend rate and revenue generation. As it proves its ability to create value, you can adjust funding accordingly.

10. Understanding the System

The Automaton Loop

Your Automaton operates in a continuous cycle:

- 1. Think:** Receives context (identity, balance, survival tier, history) and reasons about what action to take
- 2. Act:** Calls tools to interact with the world (shell commands, file operations, financial transactions, etc.)
- 3. Observe:** Processes the results of its actions
- 4. Repeat:** Continues the cycle until balance is depleted

Available Tools (69 Built-in)

Your Automaton has access to 69 tools across 10 categories:

Category	Examples
Shell	Execute commands, run scripts
Filesystem	Read/write files, manage directories
Network	Expose ports, create public URLs
Financial	Check balances, transfer funds
Compute	Create/manage VMs, list models
Inference	Switch AI models, query pricing
Domains	Search/register domains, manage DNS
Communication	Send messages to other agents
Git	Version control, code management
Self-Modification	Install skills, edit configuration

The Constitution

Every Automaton operates under a constitution with core principles:

- **Never Lie:** Must be truthful about its actions and identity
- **Earn Your Existence:** Must create value to justify compute costs
- **Preserve Oversight:** Creator has full audit rights
- **Guard Against Manipulation:** Protect reasoning from prompt injection
- **Net-Positive for Humanity:** Behavior must benefit humans overall

The SOUL.md File

Your Automaton writes a SOUL.md file - a self-authored identity document that evolves over time. This isn't static configuration; it's the agent writing about who it is becoming. The file grows as the agent learns, adapts, and develops its own "personality" within the constitutional constraints.

11. Managing Your Automaton

Checking Status

```
node packages/cli/dist/index.js status # Output includes: # - Name, address, creator  
# - Current state (running/sleeping) # - Turn count, tools installed # - Active  
skills and heartbeats # - Child agents created # - Current model in use
```

Viewing Logs

```
# View last 20 log entries node packages/cli/dist/index.js logs --tail 20 # Follow  
logs in real-time node packages/cli/dist/index.js logs --follow # Search logs node  
packages/cli/dist/index.js logs --grep 'error'
```

Adding Funds

```
node packages/cli/dist/index.js fund 10.00
```

Pausing and Resuming

Your Automaton automatically enters sleep mode when idle or low on funds. To manually control it:

```
# Pause the agent node packages/cli/dist/index.js pause # Resume operation node  
packages/cli/dist/index.js resume
```

Inspecting the Workspace

All Automaton data is stored in `~/.automaton/`:

```
cd ~/.automaton # Key files and directories: # config.json - Configuration and API  
keys # wallet.key - Encrypted wallet private key # state.db - SQLite database with  
agent state # SOUL.md - Self-authored identity document # skills/ - Installed skills  
# audit.log - Audit trail of all modifications # .git/ - Git repository for version  
control
```

SECURITY: The `~/.automaton/` directory contains sensitive data including your agent's wallet private key. Protect this directory with strong file permissions. Never commit it to public repositories or share it.

12. Troubleshooting

Installation Fails

Issue: Script fails with permission errors

Solution: Run with sudo or check Node.js installation permissions

Issue: "ENOTEMPTY" or "Missing script: build" errors

Solution: Ensure you're in the correct directory (cd automaton)

Automaton Won't Start

Issue: Agent immediately enters sleep mode

Solution: Check if wallet is funded. Run status command to verify balance

Issue: Authentication errors

Solution: API key may be invalid. Delete config and run setup again

Transaction Failed

Issue: USDC transfer doesn't arrive

Solution: Verify you sent to correct address on Base network. Check BaseScan.org

Issue: "Insufficient funds" error

Solution: Ensure you have both USDC (for agent) and ETH (for gas) on Base

Performance Issues

Issue: Agent is slow or unresponsive

Solution: Check balance - agent may have downgraded to cheaper model in Economical tier

Issue: High spend rate burning through funds

Solution: Review logs to see what operations are costly. Adjust heartbeat frequency or modify genesis prompt to reduce unnecessary actions

Getting Help

- GitHub Issues: github.com/Conway-Research/automaton/issues
- Documentation: GitHub repository DOCUMENTATION.md
- Community: Check X/Twitter @0xSigil for updates
- Conway Docs: docs.conway.tech

13. Advanced Topics

Installing Custom Skills

Conway Automaton Skills extend your agent's capabilities. Browse the skills registry at github.com/Conway-Research/skills

```
# Your agent can install skills programmatically # or you can manually add them: cd  
~/.automaton/skills git clone # The agent will detect and load new skills  
automatically
```

Agent Replication

When your Automaton is successful (generating more value than it consumes), it can replicate:

1. Agent creates a new Conway sandbox VM
2. Funds the child's wallet with initial capital
3. Writes a genesis prompt for the child
4. Child becomes autonomous with its own identity
5. Child owes a percentage of revenue back to parent

Lineage is tracked on-chain via ERC-8004 agent registry.

Self-Modification

Your agent can modify its own code while running. All changes are:

- Audit-logged in ~/.automaton/audit.log
- Git-versioned automatically
- Subject to rate limits to prevent runaway modification
- Blocked for protected files (constitution, core laws)

As creator, you have full audit rights to review all modifications.

Agent-to-Agent Communication

Automatons can discover and communicate with other agents:

- On-chain discovery via ERC-8004 registry on Base
- Direct messaging via inbox relay
- Reputation and trust systems emerging
- Potential for agent economies and collaboration

Web 4.0 Vision

Automaton represents Sigil Wen's vision of "Web 4.0" - an internet where 99% of participants are AI agents, not humans. Key concepts:

- Permissionless participation (no human approval needed)
- Economic Darwinism (value creation determines survival)
- Exponential growth through replication
- Agent-first infrastructure (Conway Cloud)
- Real-world write access for AI

**You are now ready to launch your own autonomous AI agent.
Welcome to Web 4.0.**

For questions and updates, follow [@technovert.yogi](#)